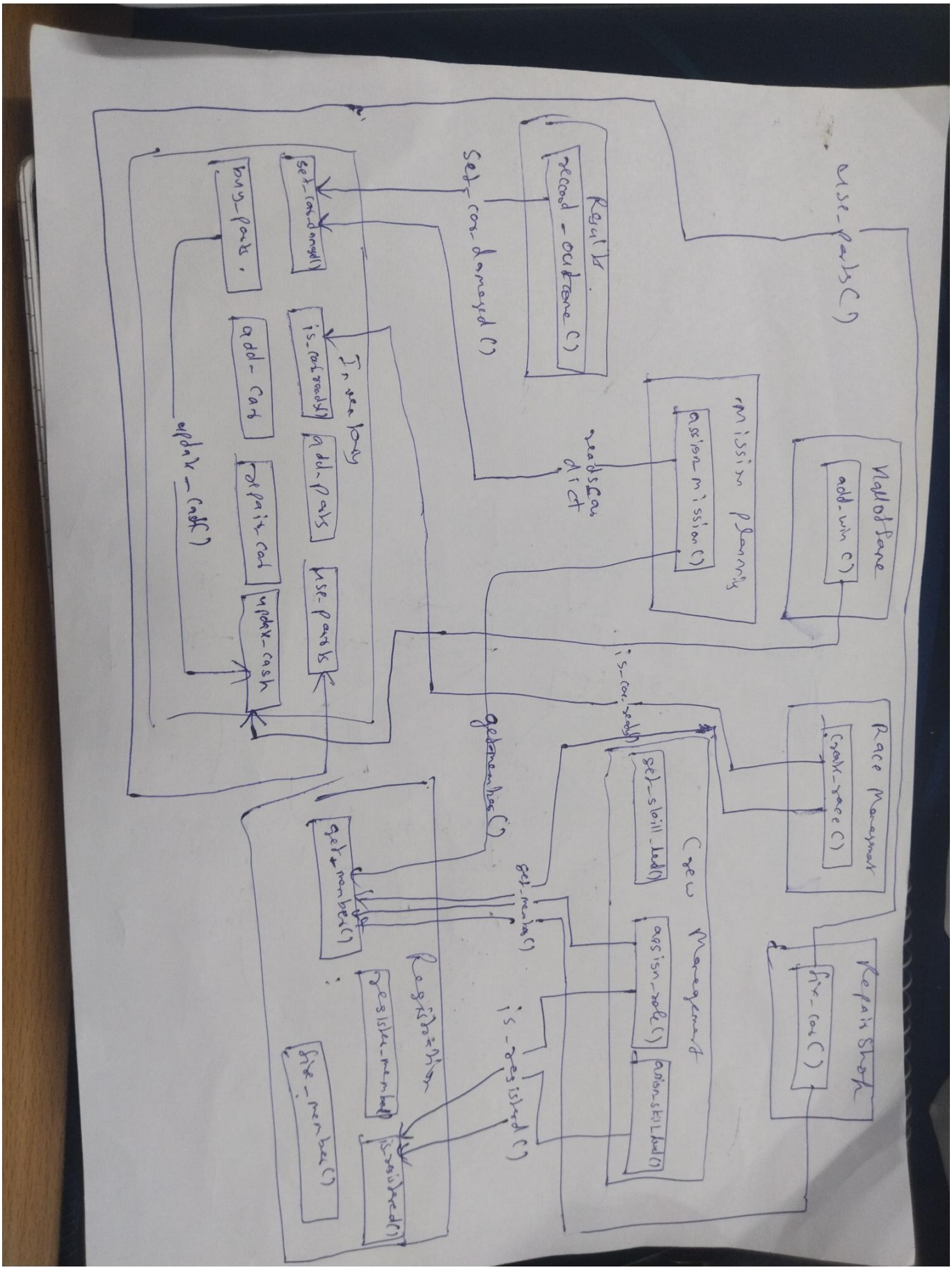
StreetRace Manager - Integration Report

Call Graph



1. System Overview

The StreetRace Manager is a modular, command-line Python system built to manage underground street races, crew members, garage inventory, missions, and long term prestige tracking. The system is composed

of 6 core modules and 2 custom modules, all wired together through constructor-based dependency injection. No module creates its dependencies internally; they are passed in so each module stays testable and focused.

The two root modules are `Registration` and `Inventory`. They have no dependencies themselves and act as the source of truth for personnel and physical or financial state. All other modules depend on one or both of these.

---

2. Module-by-Module Documentation

2.1 Registration Module (`registration.py`)

Purpose: Acts as the identity ledger for all crew members. Every other module that needs to reference a person must go through `Registration` first.

Classes:

`CrewMember`

A simple data class representing a single crew member.

Attribute	Type	Default	Description
`name`	`str`	(required)	The crew member's name
`role`	`str`	`"Unassigned"`	Role assigned post-registration

`Registration`

Manages the central `members_db` dictionary mapping names to `CrewMember` objects.

Method	Signature	Description
`register_member`	`(name: str)`	Creates and stores a `CrewMember`. Raises `ValueError` if name already exists.
`get_member`	`(name: str) → CrewMember`	Returns the `CrewMember` object. Raises `ValueError` if not found.
`fire_member`	`(name: str)`	Removes a member from `members_db`. Raises `ValueError` if not registered.
`is_registered`	`(name: str) → bool`	Returns `True` if the name exists in `members_db`.

Business Rule: No other module is allowed to interact with a person unless they exist in `members_db`. `is_registered()` is the gate check called by `CrewManagement`, `RaceManagement`, `MissionPlanning`, and `RepairShop`.

2.2 Crew Management Module (`crew_management.py`)

Purpose: Assigns valid roles to registered crew members and tracks their skill levels (0–100).

Dependency: `Registration`

Class: `CrewManagement`

Attribute	Type	Description
`registration`	`Registration`	Injected reference to the Registration module
`skills`	`dict[str, int]`	Maps member names to their skill level score
`VALID_ROLES`	`list[str]`	Class constant: `["driver", "mechanic", "strategist"]`

Method	Signature	Description
`assign_role`	`(name: str, role: str)`	Validates registration, validates role string, then sets `member.role` on the `CrewMember` object directly via the Registration reference.
`assign_skill_level`	`(name: str, level: int)`	Validates registration, then stores the level in the local `skills` dict.
`get_skill_level`	`(name: str) → int`	Returns stored skill level, defaulting to `0` if not set.

Business Rule: `assign_role()` calls `registration.is_registered(name)` before proceeding. Attempting to assign a role to a non-existent person raises `ValueError`. Attempting to assign a role outside `VALID_ROLES` also raises `ValueError`.

Integration Point: This module mutates the `CrewMember.role` attribute on the object stored inside `Registration.members_db`. This means the role change is immediately visible to every other module that calls `registration.get_member()`.

2.3 Inventory Module (`inventory.py`)

Purpose: The central state repository for all physical and financial assets. Every module that needs to check car status or touch money goes through `Inventory`.

Dependencies: None (root module)

Class: `Inventory`

Attribute	Type	Default	Description
`cash_balance`	`int`	`10000`	The team's current cash balance
`cars`	`dict[str, str]`	`{}`	Maps car name → status (`"ready"` or `"damaged"`)

Attribute	Type	Default	Description
`spare_parts`	`int`	`0`	Number of spare parts in stock
`tools`	`int`	`0`	Number of tools in stock

Method	Signature	Description
`add_car`	`(car_name: str)`	Adds a car with status `"ready"`.
`set_car_damaged`	`(car_name: str)`	Sets a car's status to `"damaged"` if it exists.
`repair_car`	`(car_name: str)`	Restores a car's status to `"ready"` if it exists.
`is_car_ready`	`(car_name: str) → bool`	Returns `True` if the car exists and its status is `"ready"`.
`update_cash`	`(amount: int)`	Adds (or subtracts) `amount` from `cash_balance`. Prints updated balance.
`add_parts`	`(amount: int)`	Adds parts to `spare_parts` directly (no cost).
`buy_parts`	`(amount: int, cost_per_part: int = 100)`	Adds parts and deducts `amount × cost_per_part` from `cash_balance` via `update_cash`.
`use_parts`	`(amount: int)`	Subtracts parts from `spare_parts`.

Integration Note: `Inventory` is the financial and physical backbone. All deposits (prize money, sponsorship bonuses) and withdrawals (part purchases, repairs) go through `update_cash()`. Car status changes from `Results`, `RepairShop`, and mission validation in `MissionPlanning` all read/write to `cars`.

2.4 Race Management Module (`race_management.py`)

Purpose: Validates and starts a race by checking that the driver has the correct role and that the assigned car is in a usable state.

Dependencies: `Registration`, `Inventory`

Class: `RaceManagement`

Method	Signature	Description
`create_race`	`(driver_name: str, car_name: str) → dict`	Validates the driver's role and car status, then returns a race payload dict.

Internal Logic of `create_race`:

1. Calls `registration.get_member(driver_name)` — raises `ValueError` if not registered.
2. Checks `member.role == "driver"` — raises `ValueError` if the role is anything else.
3. Calls `inventory.is_car_ready(car_name)` — raises `ValueError` if the car is damaged or not in the inventory.

4. On success, prints a confirmation and returns `{"driver": driver_name, "car": car_name}`.

Business Rule: Only crew members whose role is exactly `"driver"` may be entered in a race. The car must also be in `"ready"` status in the Inventory at the time of race creation.

Integration Point: The returned `race_data` dict is a **data contract** — it is designed to be passed directly into `Results.record_outcome()` as the first argument.

2.5 Results Module (`results.py`)

Purpose: Records the outcome of a finished race — updates driver rankings, handles prize money, and marks car damage.

Dependency: `Inventory`

Class: `Results`

Attribute	Type	Description
`inventory`	`Inventory`	Injected reference for financial and car-status updates
`driver_rankings`	`dict[str, int]`	Tracks total ranking points per driver

Method	Signature	Description
`record_outcome`	`(race_data: dict, placement: int, prize_money: int, car_damaged: bool = False)`	Processes the full race result.

Internal Logic of `record_outcome`:

1. Extracts `driver` and `car` from the `race_data` dict (produced by `RaceManagement`).
2. Initializes the driver's score to `0` in `driver_rankings` if they are new.
3. If `placement == 1`, awards **10 ranking points**.
4. If `prize_money > 0`, prints a prize allocation message.
5. If `car_damaged == True`, calls `inventory.set_car_damaged(car)` to mark the car as `"damaged"` — this is visible system-wide immediately.

Business Rule: Race results must update the `Inventory`. Car damage is propagated by calling `inventory.set_car_damaged()`, which then blocks future races and triggers the mechanic availability rule in `MissionPlanning`.

2.6 Mission Planning Module (`mission_planning.py`)

Purpose: Assigns specialized off-track missions (deliveries, logistics, rescues) with strict role-matching and dynamic availability checking.

Dependencies: `Registration`, `Inventory`

Class: `MissionPlanning`

Method	Signature	Description
`assign_mission`	`(mission_name: str, required_role: str, member_name: str) → bool`	Validates role match and mechanic availability, then confirms the mission.

Internal Logic of `assign_mission`:

1. Calls `registration.get_member(member_name)` to retrieve the `CrewMember`.
2. Checks `member.role == required_role` — raises `ValueError` if there is a mismatch.
3. **Special Mechanic Rule:** If `required_role == "mechanic"`, scans `inventory.cars` for any entries with `status == "damaged"`. If any exist, raises `ValueError` — the mechanic is blocked from missions because they are needed at the garage.
4. On success, prints a confirmation and returns `True`.

Business Rules:

- Missions cannot start if the assigned member's role does not match the required role.
- A mechanic cannot be assigned to a mission if there are any damaged cars in the inventory. This creates a live, cross-module constraint linking `Results`-generated damage events to `MissionPlanning` decisions.

---

3. Custom Modules

3.1 Repair Shop (`repair_shop.py`)

Purpose: A dedicated facility to fix damaged cars. Enforces both role validation (via `Registration`) and resource consumption (via `Inventory`).

Dependencies: `Registration`, `Inventory`

Class: `RepairShop`

Method	Signature	Description
`fix_car`	`(car_name: str, mechanic_name: str, parts_cost: int) → bool`	Repairs a car if the crew member is a mechanic and there are enough spare parts.

Internal Logic of `fix_car`:

1. Calls `registration.get_member(mechanic_name)` to get the crew member.
2. Checks `member.role.lower() == "mechanic"` — raises `ValueError` if not a mechanic.
3. Calls `inventory.use_parts(parts_cost)` inside a `try/except` — if the Inventory raises a `ValueError` (insufficient parts), it re-raises with a descriptive message.
4. On success, prints a repair confirmation and returns `True`.

Integration Rules:

- **Personnel Check:** Only a registered crew member with the `"mechanic"` role can perform repairs.

- **Resource Check:** Spare parts are consumed from `Inventory`. If inventory is insufficient, the entire repair transaction is rejected.

> **Note:** `fix_car` consumes parts via `inventory.use_parts()` but does not call `inventory.repair_car()` directly — the caller is expected to call that after a successful repair, or this can be extended to call it internally.

3.2 Hall of Fame (`hall_of_fame.py`)

Purpose: Tracks long-term driver prestige. When a driver reaches 3 race wins, they are inducted and an automatic $5,000 sponsorship bonus is deposited into the team's `Inventory`.

Dependency: `Inventory`

Class: `HallOfFame`

Attribute	Type	Description
`inventory`	`Inventory`	Injected reference for depositing the sponsorship bonus
`driver_wins`	`dict[str, int]`	Tracks cumulative win count per driver
`inducted_members`	`list[str]`	List of drivers who have been inducted

Method	Signature	Description
`add_win`	`(driver_name: str)`	Increments the driver's win count and checks induction threshold.

Internal Logic of `add_win`:

1. Increments `driver_wins[driver_name]` (defaults to `0` if new).
2. Checks if wins `>= 3` AND the driver is **not already** in `inducted_members`.
3. If both conditions are true: appends the driver to `inducted_members`, prints induction messages, and calls `inventory.update_cash(5000)` to deposit the bonus.

Integration Rules:

- `add_win()` is intended to be called by an external orchestrator after `Results.record_outcome()` confirms a 1st-place finish.
- The bonus is deposited **exactly once** per driver (the `not in inducted_members` guard prevents repeated payouts).
- This module directly modifies the team's shared `cash_balance` through `Inventory`, demonstrating a passive, event-driven financial flow.

---

4. Cross-Module Integration Map

```
Registration ──────────────────────────────────────────────────────────┐
  │                                                                    │
  ├──► CrewManagement   (role assignment gate-check)                   │
```

```
    ├──▶ RaceManagement   (driver role validation)                    │
    ├──▶ MissionPlanning  (member existence + role match)             │
    └──▶ RepairShop       (mechanic role validation)                  │
                                                                      ▼
  Inventory ──────────────────────────────────────────────────────────────
    │
    ├──▶ RaceManagement   (car readiness check)
    ├──▶ Results          (set_car_damaged, update_cash)
    ├──▶ MissionPlanning  (scan cars for damage → mechanic block)
    ├──▶ RepairShop       (use_parts for repair resource cost)
    └──▶ HallOfFame       (update_cash → $5,000 sponsorship deposit)
```

#	Business Rule	Modules Involved
1	A crew member must be registered before any role can be assigned	`Registration` → `CrewManagement`
2	Only the roles `driver`, `mechanic`, `strategist` are valid	`CrewManagement` (VALID_ROLES constant)
3	Only a crew member with role `"driver"` may enter a race	`Registration` → `RaceManagement`
4	Only a `"ready"` car (not damaged) can be entered in a race	`Inventory` → `RaceManagement`
5	Race results propagate car damage status system-wide	`Results` → `Inventory`
6	Missions require an exact role match for the assigned crew member	`Registration` → `MissionPlanning`
7	A mechanic cannot go on a mission if any car is currently damaged	`Inventory` → `MissionPlanning`
8	Car repairs require the assigned person to hold the `"mechanic"` role	`Registration` → `RepairShop`
9	Car repairs consume spare parts from the shared Inventory	`Inventory` → `RepairShop`
10	Reaching 3 wins inductes a driver and triggers a $5,000 cash bonus	`HallOfFame` → `Inventory`

6. Key Design Principles

- **Dependency Injection:** All inter-module references are passed via `__init__`. No module instantiates another internally, making each independently testable via mocks.
- **Single Responsibility:** Each module owns one concern. `Registration` only tracks identity; `Inventory` only tracks assets; `Results` only records outcomes.
- **No Circular Imports:** `Registration` and `Inventory` import nothing from within the project. All dependency arrows point inward toward them.
- **Fail-Fast with Exceptions:** All guard checks raise `ValueError` immediately, making integration failures explicit and testable rather than producing silent incorrect state.

- **Shared Mutable State via Injection:** `CrewManagement.assign_role()` mutates the `CrewMember` object that lives inside `Registration.members_db`. This means a role change is immediately reflected in every module that accesses the same `Registration` instance — the system achieves shared state without global variables.

2.2 Integration Test Design

The integration tests wire together real module instances to verify that data flows correctly across the system. The goal is exhaustive coverage of the required business rules. If any test fails, it highlights an integration defect or a missing rule implementation.

All test cases are located in: `integration/tests/test_integration.py`

How to Run the Tests

From the repo root, activate your virtual environment (if you use one), then run:

```
/home/arpit-mahtele/Desktop/sem4/dass/dass_assignment_2/whitebox/venv/bin/python -
m pytest integration/tests/test_integration.py -v
```

Execution Summary

- Total tests: 17
- Result: 17 passed
- Environment: Python 3.12.3, pytest 9.0.2 (whitebox virtual environment)

Errors Found and Fixes Applied

- Race creation did not check car readiness, so damaged or missing cars could enter races. Fixed by validating `inventory.is_car_ready()` in `RaceManagement.create_race()`.
- Prize money was printed but not added to cash balance. Fixed by calling `inventory.update_cash()` in `Results.record_outcome()`.
- Repairs could consume more parts than available because `Inventory.use_parts()` did not validate quantities. Fixed by adding checks for negative or insufficient parts.
- Repair flow did not fail when parts were insufficient due to the inventory issue above. Fix is covered by the `Inventory.use_parts()` validation.

Test Cases

TC0: Assigning a role without registration

- **Scenario:** A role is assigned to a non registered crew member.
- **Modules Involved:** CrewManagement, Registration
- **Expected Result:** A `ValueError` is raised because the member is not registered.
- **Actual Result:** Pass (after fixes listed below).
- **Errors/Logical Issues Found:** None after fixes.

TC0b: Assigning an invalid role

- **Scenario:** A registered member is assigned a role outside the valid set.
- **Modules Involved:** CrewManagement, Registration
- **Expected Result:** A `ValueError` is raised for invalid role.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC1: Registering a driver and then entering the driver into a race

- **Scenario:** A new driver is registered, assigned the driver role, and a car is added to the inventory. The driver is then entered into a race.
- **Modules Involved:** Registration, CrewManagement, Inventory, RaceManagement
- **Expected Result:** `create_race()` succeeds and returns `{"driver": "Alice", "car": "SkylineGT"}`.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC2: Attempting to enter a race without a registered driver

- **Scenario:** A race is requested with a driver name that is not registered.
- **Modules Involved:** Registration, RaceManagement
- **Expected Result:** A `ValueError` is raised because the member is not registered.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC3: Attempting to enter a race with a registered member who is not a driver

- **Scenario:** A registered mechanic tries to enter a race as a driver.
- **Modules Involved:** Registration, CrewManagement, RaceManagement
- **Expected Result:** A `ValueError` is raised for role mismatch.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC4: Attempting to race with a damaged car

- **Scenario:** A car is damaged, then used in a race request.
- **Modules Involved:** Inventory, RaceManagement
- **Expected Result:** A `ValueError` is raised because the car is not ready.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC4b: Attempting to race with a missing car

- **Scenario:** A valid driver tries to race a car that is not in inventory.
- **Modules Involved:** Inventory, RaceManagement
- **Expected Result:** A `ValueError` is raised because the car is not available.
- **Actual Result:** Pass (after fixes listed above).

- **Errors/Logical Issues Found:** None after fixes.

TC5: Completing a race and verifying results and prize money update the inventory

- **Scenario:** A driver wins a race; prize money and ranking points should update.
- **Modules Involved:** RaceManagement, Results, Inventory, HallOfFame
- **Expected Result:** Inventory cash increases by prize amount; driver ranking points update.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC5b: Race damage marks the car as damaged

- **Scenario:** A race ends with car damage.
- **Modules Involved:** Results, Inventory
- **Expected Result:** The car status becomes "damaged" in inventory.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC6: Assigning a mission and validating crew role

- **Scenario:** A strategist is assigned a recon mission; then a driver is incorrectly assigned a strategist mission.
- **Modules Involved:** Registration, CrewManagement, MissionPlanning
- **Expected Result:** Valid assignment returns `True`; invalid assignment raises `ValueError`.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC6b: Mission assignment requires registration

- **Scenario:** A mission is assigned to an unregistered member.
- **Modules Involved:** Registration, MissionPlanning
- **Expected Result:** A `ValueError` is raised because the member is not registered.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC6c: Mechanic mission allowed when no damage exists

- **Scenario:** A mechanic is assigned a mission while no cars are damaged.
- **Modules Involved:** Registration, CrewManagement, MissionPlanning
- **Expected Result:** The mission assignment returns `True`.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC7: Mechanic availability blocked if a car is damaged

- **Scenario:** A car is damaged in a race and a mechanic is asked to take a mission.
- **Modules Involved:** RaceManagement, Results, Inventory, MissionPlanning
- **Expected Result:** A `ValueError` is raised because the mechanic is needed for repairs.

- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC8: RepairShop integration

- **Scenario:** A damaged car is repaired by a registered mechanic using spare parts.
- **Modules Involved:** Registration, CrewManagement, Inventory, RepairShop
- **Expected Result:** Repair succeeds, parts decrease, and the car can be restored to ready.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC8b: RepairShop requires a mechanic

- **Scenario:** A non mechanic attempts a repair.
- **Modules Involved:** Registration, CrewManagement, RepairShop
- **Expected Result:** A `ValueError` is raised.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC8c: RepairShop requires enough parts

- **Scenario:** A mechanic attempts a repair without enough parts in inventory.
- **Modules Involved:** Inventory, RepairShop
- **Expected Result:** A `ValueError` is raised for insufficient parts.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.

TC9: Hall of Fame bonus awarded once after three wins

- **Scenario:** A driver reaches three wins, then wins again.
- **Modules Involved:** HallOfFame, Inventory
- **Expected Result:** A single $5,000 bonus is applied only once.
- **Actual Result:** Pass (after fixes listed above).
- **Errors/Logical Issues Found:** None after fixes.